

1. 字符串类型

在C语言中,没有字符串类型的。C语言中使用字符数组/字符指针来表示一串字符

```
char str[128] = {"hello"}; //栈区, 可以修改
str[0] = 'H'; //right
char *p_str = "hello"; //常量区
p_str[0] = 'H'; //error 常量区的内容不能修改
```

在 C++ 标准库中,提供了一个字符串类型,实际上是一个字符串类,类名叫做 `string`,专门用于描述一个字符串对象,并且提供了操作字符串的一系列函数接口。

- 使用方法

指定作用域, 因为是标准库中的, 所以要使用 `std` 的命名空间。

事实上 `string` 并不是类型, 与 `int`、`char` 这些不同, 而是一个类, 封装在标准库中, 因此使用的时候要加 `std::`。而平常我们可以使用的 `+` `>` 等运算符操作字符串是因为其内部重载了这些运算符。能打印出来是因为重载

```
std::string str = "蔡徐坤";
```

- 字符串对象的实例化方式

- 无参构造函数方式

```
std::string str; //空字符串
```

- 含参构造方式

```
std::string str("蔡徐坤");
或者
std::string str = std::string("范丞丞");
```

//使用字符和长度作为参数的构造函数:

```
std::string str2("abcdefg", 3);
```

- 拷贝构造

```
std::string str1 = std::string("范丞丞");
std::string str2(str1);
//std::string str2 = str1;
```

//当然也可以使用字符数组作为拷贝构造的参数

```
char c[] = "范丞丞";
//std::string str2(c);
std::string str2 = c;
```

- 拷贝赋值

```
std::string str1 = std::string("范丞丞");  
std::string str2("蔡徐坤");  
str2 = str1;
```

//当然也可以使用字符数组作为拷贝赋值的对象

```
char c[] = "范丞丞";  
std::string str2("蔡徐坤");  
str2 = c;
```

- 常用的成员函数（访问元素）

- `at(int size)`

访问指定字符,有边界检查, 返回指定位置的字符的引用

```
std::string str("abcdefg");  
char c = str.at(3); //d 如果越界会崩溃  
std::cout << c << std::endl; //d
```

- `operator[]` 访问指定字符

即下标访问

```
std::string str("abcdefg");  
char c = str[3]; //d  
std::cout << c << std::endl;
```

- `front()`

访问首字符

```
std::string str("abcdefg");  
char c = str.front(); //a  
std::cout << c << std::endl;
```

- `back()`

访问最后有效字符

```
std::string str("abcdefg");  
char c = str.back(); //g  
std::cout << c << std::endl;
```

- `data()`

返回指向字符串首字符的指针

```
std::string str("abcdefg");  
const char *p = str.data();  
std::cout << p << std::endl; //abcdefg  
  
std::cout << std::hex << (void *)p << std::endl; //地址
```

- `c_str()`

返回指向字符串首字符的指针,同 data()

```
std::string str("abcdefg");
const char *p = str.c_str();

std::cout << p << std::endl;
```

- 容量

- empty 检查字符串是否为空, 返回布尔值

```
string str = "abcdefg";
cout << str.empty() << endl;
```

- size / length 返回有效字符长度

与sizeof()区分, sizeof是分配的长度, size / length 是实际使用的长度

```
string str = "abcdefg";
cout << str.size() << endl; // 7
cout << str.length() << endl; // 7
cout << sizeof(str) << endl; // 40
```

- max_size 返回 std::string 类型对象所能容纳的最大字符数。
(9223372036854775807)

需要注意的是, max_size() 返回的是一个理论上的最大值, 并不意味着您的计算机能够实际分配和存储这么多字符。实际上, max_size() 返回的值可能受到系统或编译器的限制, 例如可用内存大小或其他资源限制。但一般是不会越界的

```
string str = "abcdefg";
cout << str.max_size() << endl;
```

- reserve(size_t size) 是一个成员函数, 用于预留字符串对象的内存空间, 以容纳指定数量的字符。

```
string str = "abcdefg";
str.reserve(100); // 预留至少能容纳 100 个字符的内存空间
```

- capacity 用于返回 std::string 对象当前已分配的内存空间可以容纳的最大字符数。

capacity() 函数返回的是一个无符号整数 (size_t 类型), 表示 std::string 对象当前已分配的内存空间的大小。注意, capacity() 返回的值可能大于 std::string 对象的长度 (length() 或 size() 返回的值), 因为 std::string 对象可能预留了额外的内存空间以便于添加更多的字符。

```
string str = "abcdefg";
std::cout << "String length: " << str.length() << std::endl; // 字符串长度
std::cout << "String capacity: " << str.capacity() << std::endl; // 字符串内存空间大小
```

- 字符串操作

- clear 清除内容

```
std::string str = "Hello";

str.clear();

cout << "str = " << str << endl;
```

- `insert(size_t size, 字符串)` 插入字符串

```
std::string str = "Hello";

str.insert(1, "abc");

cout << "str = " << str << endl; //Habcello
```

- 查找

- `find(字符串)`，函数接受一个字符串或字符作为参数，并返回第一次出现的位置的索引。如果没有找到，它会返回一个特殊的值 `std::string::npos`。

```
std::string str = "abcdefg";

if (str.find("def") != std::string::npos)
{
    cout << str.find("def") << endl;
}
```

2. 迭代器模式

2.1 设计模式

设计模式是一种在软件设计中常用的解决问题的方案或模板。它们描述了在特定情境下的问题和解决方案，并提供了一种被广泛接受的方法来设计和构建软件系统。

设计模式可以帮助开发人员解决常见的设计问题，并提供一种可重用的、经过验证的方法来构建高质量的软件系统。它们被视为一种优秀的实践，可以提高代码的可维护性、可扩展性和可重用性。

设计模式并不是一种必须遵循的规则，而是一种经验总结和共享的最佳实践。开发人员可以根据具体的问题和需求选择合适的设计模式，并根据实际情况进行适当的调整和定制。

需要注意的是，设计模式并非适用于所有情况。在使用设计模式时，开发人员应该根据实际需求和系统复杂性进行评估，并权衡使用设计模式所带来的好处和开销。

2.2 迭代器模式

在软件开发的过程中，集合内部的结构经常发生改变。对于这些集合对象（用户其实不关心集合内部是如何实现的，只要能正常使用就可以了）用户希望在可以不了解（不暴露）内部结构的同时，可以让外界透明的访问其中的元素，不管集合内部的数据结构如何变化，反正对于集合外部的访问接口都保持不变。

这种“**让外界透明的访问**”为同一种接口（`begin/end`）可以在多种集合对象上进行同一种操作。

使用面向对象的技术，讲这种遍历机制抽象为“**迭代器对象**”，可以描述、表示一个元素的位置为遍历“**变化中的集合对象**”，提供一种不变的访问接口。

设计模式中对迭代器模式的定义如下：

- 容器（集合）应该提供一种方法。顺序的访问一个集合对象中的各个元素，而又不暴露该对象的内部表示!!!

容器就是一个集合，可以理解为存储很多元素的一个集合
如之前所学的数组、字符串等都属于容器
本文只讨论基本类型数组和字符串

C++的一般做法：

把遍历集合使用的“指针”，封装成一个迭代器类型，并且迭代器类型向外提供相对应的接口（++，+=，--，*，->...），并且集合对象也应该向外提供获取迭代器的接口（begin/end）

```
class Iterator
{
private:
    // 指向元素的类型的 指针 ， 假定为T类型
    T* m_p;
public:
    // 构造函数
    // 析构函数
    // ...
    next(); // 让当前迭代器表示当前条目的下一个条目...
    // +=
    // ==
    // !=
    // ...
};
```

把迭代器对象完成之后，容器应该向外部提供一组获取迭代器的接口

- `Iterator begin();`
- `Iterator next();`
- `Iterator end();`

2.2.1 迭代器的定义

- 基本类型（和指针的定义方法一致）

类型 *迭代器变量名；

举例

```
int *iter;
```

- 容器类型（以string为例，后续会详细学容器）

```
string::iterator iter;
```

2.2.2 迭代器常用函数

- `begin()`

返回值：容器（集合）的首元素的地址

- 基本类型

```
std::begin(数组名);
```

举例

```
int arr[5] = { 1, 2, 3, 4, 5 };  
std::begin(arr);
```

- 容器类型（以string为例）

```
字符串变量名.begin();
```

举例

```
std::string str("abcdefg");  
str.begin();
```

- `end()`

返回值：容器（集合）的最后一个元素的下一个地址

- 基本类型

```
std::end(数组名);
```

举例

```
int arr[5] = { 1, 2, 3, 4, 5 };  
std::end(arr);
```

- 容器类型（以string为例）

```
字符串变量名.end();
```

举例

```
std::string str("abcdefg");  
str.end();
```

2.2.3 使用迭代器遍历基本类型的数组

与指针遍历原理相似，都是通过地址偏移来完成。

`iter != std::end(arr)` 是因为`end()`函数返回最后一个有效数字地址的下一个地址
对于所有的迭代器来说，迭代器的类型都可以让系统自己去推断，在类型处定义为`auto`即可。
尽管如此，建议初学者还是明确写出迭代器类型。

```

#include<iostream>
using namespace std;
int main()
{
    int arr[5] = { 1, 2, 3, 4, 5 };
    //iter != std::end(arr)是因为end()函数返回最后一个有效数字地址的下一个地址
    //for (int * iter = std::begin(arr); iter != std::end(arr); iter=
    std::next(iter))
        for (auto iter = std::begin(arr); iter != std::end(arr); iter =
        std::next(iter))
        {
            cout << *iter << endl;
        }

    for (int* p = arr; p < arr + 5; p++)
    {
        cout << *p << endl;
    }
    return 0;
}

```

几个可能会出现疑问。

- 迭代器遍历和指针遍历完全相同，为什么使用迭代器而不使用指针？

迭代器的遍历和指针的遍历在功能上是相同的，都可以用来遍历数组或其他容器的元素。事实上，迭代器在很大程度上就是对指针的封装和抽象。

然而，使用迭代器而不直接使用指针有以下几个优势：

1. 抽象性和通用性：迭代器提供了一种通用的遍历方式，不仅适用于数组，还适用于其他各种容器，如链表、集合、映射等。它们都可以使用统一的迭代器接口进行遍历，使得代码更加通用和可移植。
2. 安全性和封装性：迭代器可以提供更好的安全性和封装性。迭代器隐藏了具体容器的实现细节，只暴露必要的操作接口，从而降低了使用者的错误风险。此外，迭代器还可以提供一些额外的保护机制，例如防止迭代器越界访问容器的元素。
3. 可迭代性的概念：使用迭代器遍历容器的语法更符合可迭代性的概念。通过使用迭代器，我们可以使用类似于 `begin()`、`end()`、`++`、`*` 等语法来进行遍历，使代码更加清晰和易读。而使用指针来遍历则可能需要使用更多的指针算术运算和条件判断，增加了代码的复杂性。

总结来说，尽管迭代器的遍历和指针的遍历在功能上是相同的，但迭代器提供了更抽象、更通用、更安全和更符合可迭代性概念的方式来遍历容器元素。因此，在实际编程中，使用迭代器而不是直接使用指针可以提高代码的可读性、可维护性和可扩展性。

在上述的案例中其实就可以发现一些端倪。当使用指针遍历是，循环终止条件是 `p < arr + 5`，即 `数组名称+数组长度`，该方法有问题，如果当数组长度不确定时，就有可能出现数组越界风险，而且不同数组长度不同，循环条件也不相同，必须针对不同的数组改变循环条件。但是使用迭代器就完全避免了这个问题的出现。尽管如此，基本类型的数组建议大家使用指针遍历。

- 循环终止条件为什么是 `iter != std::end(arr)`

因为 `end()` 函数返回最后一个有效数字地址的下一个地址

- 为什么 `end()` 函数返回值是最后一个有效数字地址的下一个地址，而不直接是最后一个有效数字的地址

`end()` 函数返回的是指向容器中最后一个有效元素之后的位置，而不是指向最后一个有效元素本身的地址。

这是因为在大多数编程语言中，使用半开区间（half-open interval）的方式来表示范围，即左闭右开。这意味着，范围是从左边界开始（包括左边界），到右边界结束（不包括右边界）。

对于容器来说，`begin()` 函数返回的是指向第一个元素的地址，而 `end()` 函数返回的是指向最后一个元素之后的位置的地址。这样设计的原因是为了方便使用迭代器进行遍历，当迭代器指向 `end()` 返回的位置时，表示已经到达容器的末尾。

以数组为例，假设数组中有5个元素，索引从0到4。`begin()` 返回的是指向索引为0的元素的地址，而 `end()` 返回的是指向索引为5的位置的地址，即指向数组范围之外的位置。这样，我们可以使用迭代器 `it` 来遍历数组，当 `it` 指向 `end()` 返回的位置时，表示已经遍历到了数组的末尾。

使用半开区间的方式可以避免边界条件的复杂性和歧义，并且在处理空容器时也更加方便，因为 `begin()` 和 `end()` 返回的位置是一致的，表示空范围。

2.2.4 使用迭代器遍历容器（字符串）

迭代器（是一种思想,是为了让用户更加方便的访问容器中的元素而提供的接口），可以让用户在不需要了解容器内部结构的情况下,很方便的访问容器中的元素

- 迭代器对象：可以看做是指向容器中用户数据的一个位置

字符串提供了一系列的接口,可以获取某些特殊位置的迭代器，把迭代器看做是一个表示位置的对象

- `begin` 返回指向容器第一个元素的迭代器(位置)
- `end` 返回指向容器最后一个元素后面的迭代器

同上述的迭代器遍历数组类似

```
#include<iostream>
//#include<iterator>
using namespace std;
int main()
{
    string str("abcdefg");
    for (string::iterator iter = str.begin(); iter != str.end(); iter++)
    {
        cout << *iter << endl;
    }
    return 0;
}
```

使用 `string::iterator` 迭代器有可能意外修改字符串的值，如果不希望修改字符串的内容，可以定义为常迭代器。

通常和 `cbegin` 和 `cend` 搭配使用，防止修改容器的值，`cbegin`、`cend` 的返回值和 `begin`、`end` 相同

```
string::const_iterator iter;
```


- `c:const` : 不可以通过迭代器修改元素
 - `cbegin` 返回指向容器第一个元素的常量迭代器(位置)
 - `cend` 返回指向容器最后一个元素后面的常量迭代器, 不能通过迭代器去修改迭代器指向的位置的数据, 只能访问

```
#include<iostream>
//#include<iterator>
using namespace std;
int main()
{
    string str("abcdefg");
    for (string::const_iterator iter = str.cbegin(); iter != str.cend(); iter++)
    {
        /*iter = 'q';
        cout << *iter << endl;
    }
    return 0;
}
```

使用指针反向遍历字符串通常需要将指针指向最后一个有效数字位置, 然后递减指针的值打印出来。

在容器中使用反向迭代器来实现

```
string::reverse_iterator iter;
string:: const_reverse_iterator iter;
```

- `r:reserve` 反向的
 - `rbegin` 返回指向容器最后一个元素的反向迭代器
 - `rend` 返回指向容器第一个元素前面的反向迭代器, `++` 的时候是往前移动的, `--` 的时候是往后移动的
- `crbegin` 返回指向容器最后一个元素的常量反向迭代器
- `crend` 返回指向容器第一个元素前面的常量反向迭代器

```
#include<iostream>
//#include<iterator>
using namespace std;
int main()
{
    string str("abcdefg");
    for (string::reverse_iterator iter = str.rbegin(); iter != str.rend(); iter++)
    {
        /*iter = 'q';
        cout << *iter << endl;
    }
    for (string::const_reverse_iterator iter = str.rbegin(); iter != str.rend();
    iter++)
    {
        /*iter = 'q';
        cout << *iter << endl;
    }
    return 0;
}
```

```
}
```

o erase() 删除字符串

- 当参数是整数型时，删除该位置后面所有字符串
- 当参数是迭代器时，删除当前迭代器位置的字符，并返回下一个位置的迭代器

```
string str = "abc defg";  
str.erase(5);  
cout << "str = " << str << endl;  
for (auto it = str.begin(); it != str.end(); ) {  
    if (*it == ' ') {  
        it = str.erase(it); // 删除当前位置的空格字符，并返回下一个位置的迭代器  
    }  
    else {  
        ++it;  
    }  
}  
cout << "str = " << str << endl;
```